

Declarative Capability Language (DCL)

Invitation for Critical Review

What is DCL?

DCL is an implementation-independent declarative programming language for describing architectural behaviour.

Rather than modelling systems as services, APIs, classes or infrastructure, DCL models them in terms of business capabilities, intent, outcomes, invariants, policies, effects and lifecycle.

Its purpose is to represent architectural intent explicitly, allowing it to be validated, analysed and understood by both humans and machines.

Why was it created?

Software architecture is typically communicated through documents, diagrams, ADRs and conversations.

These are valuable communication tools but are difficult to validate automatically because much of the meaning remains implicit.

DCL explores whether architectural behaviour can instead be represented as a compiler-validated semantic model.

Current State

DCL v1.0 currently includes:

- Declarative language
 - Compiler
 - Intermediate Representation (IR)
 - Semantic validation
 - Analysis passes
 - VS Code extension
 - MCP server
 - Documentation
 - Reference website
 - Working examples
-

Unexpected Observation

DCL was not designed for AI.

However, large language models appear to reason over DCL considerably more effectively than equivalent architectural prose.

This has raised new questions around semantic representations for AI-assisted engineering.

Areas Where Feedback Is Requested

I would particularly appreciate informed critique on the following questions.

1. Does DCL represent a genuinely useful abstraction for expressing architectural intent independently of implementation?
 2. Which existing research, languages or modelling approaches should DCL be compared against?
 3. Which aspects would require stronger formalisation before DCL could be considered a meaningful research contribution?
-

Resources

Website: <https://russelleast.github.io/Capability-Language/>

Github: <https://github.com/russelleast/Capability-Language>

Compiler & Tooling: <https://russelleast.github.io/Capability-Language/tooling/>

About the Author

Russell East is a Principal Software Architect with 30 years of software engineering experience spanning enterprise systems, distributed architectures and cloud platforms.

DCL emerged from practical architectural work rather than academic research, motivated by the challenge of expressing architectural intent in a precise, implementation-independent form. The language has since evolved into a standalone compiler, tooling ecosystem and open-source project.

Russell is now seeking critical feedback from researchers and practitioners to help evaluate DCL's place within existing programming language, software architecture and AI engineering research.

Why you?

You are receiving this document because your work relates to one or more of the areas that DCL touches: programming languages, software architecture, formal methods, or AI-assisted software engineering. I would value your perspective on where DCL aligns with existing work, where it differs, and where its ideas could be strengthened.